

NICOLE CHIA QI LEE

2nd year Imperial College London Computing Student

About Me

A driven and collaborative Computer Science student with a passion for building impactful software solutions. I thrive in team environments and enjoy leveraging my technical skills to solve complex real-world problems. I am committed to developing user-centric systems that provide tangible benefits to others, constantly seeking to refine my craft and embrace new challenges in software engineering.

Feel free to visit my portfolio to know more about me :
<https://nicoleleechiaqi.github.io/Portfolio/>

Contact Info

nicoleleecq@gmail.com
+44 7887 003 656
+6011 5989 4412

Skills & Proficiencies

- Programming
 - Haskell
 - Kotlin
 - React & Typescript
 - Python
- Soft Skills
 - Leadership
 - Vice Head Prefect (2022)
 - Communications
 - Problem-solving
 - Adaptability

Education

- **BEng Computing (2nd Year)**
Imperial College London Current
expected graduation month/year : June 2028
 - **A-Level**
Kolej Yayasan UEM (KYUEM) 2023 - 2025
 - **Sijil Pelajaran Malaysia (SPM)**
SMJK Sin Min 2018 - 2022
- Academic Result**
- BEng Computing : expected first class*
A-Level : 3A 1A*
Sijil Pelajaran Malaysia (SPM) : 11 A+

Experience

Job Experience

- **Intern**
OpenLaunch start-up company June 2025 - July 2025
- National AI Competition (NAIC) Voting Platform**
 - Collaborated in a cross-functional team to develop a high-traffic voting webpage for the NAIC, successfully managing over 1,000 unique voters.
 - Engineered a responsive UI using Next.js and React, ensuring seamless deployment and scalability via Vercel.
 - Implemented secure, cookie-based tracking and integrated Qdrant to maintain data integrity and prevent duplicate voting.
- CV Semantic Search Engine**
 - Developed a semantic search tool to identify the top 10 most compatible candidates based on job requirements, utilizing Gemini Flash embeddings.
 - Optimized search accuracy by architecting a Qdrant vector database and implementing cosine similarity with custom-weighted scoring for different CV sections.
 - Designed a custom text-chunking pipeline and a keyword-matching interface to highlight candidate compatibility features.

Projects

● Number Recognition

- Built PyTorch Data Pipeline: Implemented data loaders with normalization and geometric augmentations (affine transforms, rotations) to enhance model generalization.
- Engineered Image Preprocessing Pipeline: Designed an OpenCV/SciPy pipeline using color inversion, aspect-ratio resizing, and center-of-mass alignment to normalize real-world handwriting.
- Developed Live Inference GUI: Created an interactive Tkinter/PIL drawing canvas enabling real-time stroke capture, pipeline integration, and low-latency digit prediction with confidence scores.
- Constructed Model Evaluation Suite: Automated classification reporting (Precision, Recall, F1-Score) and confusion matrix visualization using Scikit-Learn, Seaborn, and Matplotlib.

Projects

● Fitness & Health Website

- Full-Stack Architecture: Built an end-to-end health platform using React, TypeScript, and Firebase, combining generative AI capabilities with dynamic wellness tracking.
- Multimodal AI Integration: Engineered an automated meal scanner with the Gemini API to convert food photos and user context into strictly typed JSON nutritional estimates and ingredient breakdowns.
- Conversational AI Assistant: Developed an interactive health chatbot featuring multi-turn conversation memory, prompt orchestration, and optimistic UI updates for immediate user feedback.
- Auth & Cloud Persistence: Integrated Google OAuth via Firebase Auth and structured user onboarding and vitals profiles using Firebase Realtime Database.
- Client Automation & APIs: Built an interval-based medication scheduler with automated midnight resets, paired with OpenWeather and Geolocation APIs for real-time workout safety recommendations.

● Dream Studio

- Built a full-stack sleep application with React, TypeScript, and Google Colab, leveraging F5-TTS via Gradio Client for zero-shot voice cloning from 5–8 second audio samples.
- Engineered a client-side audio pipeline using Web Audio API to chunk long scripts, synthesize parts sequentially, and assemble seamless WAV files.
- Implemented synchronized dual-track playback pairing HTML5 voice audio with a headless YouTube player for ambient background mixing, complete with an automated 30-second sleep fade-out.
- Architected an offline-first storage library using IndexedDB to cache synthesized audio blobs and metadata for instant local playback and file export.

● Mario Game

- Developed an arcade-style platformer game in C using the Raylib library, featuring dynamic side-scrolling, tile-based level rendering, and modular state management.
- Engineered custom 2D physics engines including gravity, acceleration, terminal velocity, and sub-step collision detection with grid alignment to prevent tunneling through solid tiles.
- Designed an object pooling system to manage dynamic entities, including enemy patrol behaviors, stomping mechanics, and hazard-state warp pipes with animated obstacles.
- Integrated multi-source hardware and software input handling supporting keyboard controls alongside low-level Raspberry Pi GPIO pin reading for arcade cabinet interfacing.
- Built full gameplay lifecycle loops incorporating progressive difficulty scaling, audio stream management, invulnerability frames, score tracking, and persistent state transitions.

● Sudoku Solver

- Functional Constraint Solver: Built a pure Haskell solver utilizing constraint propagation (worklists/pencil-marking) paired with MRV heuristic backtracking to solve puzzles efficiently.
- Custom Data Structure: Designed and implemented a balanced ternary tree (Tria) to enable efficient immutable grid indexing and functional state updates.
- Interactive Terminal UI: Created a terminal interface using ANSI escape sequences for dynamic grid rendering, cursor highlighting, and live status feedback.
- Low-Level Terminal Control: Managed raw terminal I/O to parse multi-byte arrow key sequences and drive an event-driven interactive loop.

● Cartoonise Image

- Computer Vision Pipeline: Built an image cartoonization engine in Python (OpenCV, NumPy) using edge-preserving bilateral filtering, color quantization, and ink outline extraction.
- Palette Quantization: Implemented K-Means clustering (CIELAB) and HSV binning to simplify gradients, accelerated with Gaussian pyramid scaling.
- Outline Extraction: Designed an edge detection module combining median filtering with Adaptive Thresholding and Canny for hand-drawn ink line masks.
- Real-Time GUI Studio: Created a live Tkinter and OpenCV control panel with style presets, webcam processing, and state-cached rendering to prevent redundant compute.